Student Record Search System

Binary Search Tree (BST) — Project Report

Introduction

The Student Record Search System is a menu-driven C++ program that stores and manages student records using a **Binary Search Tree (BST)**. Each record contains an enrolment ID, student name, and department. The enrolment ID is used as the key for organizing the tree, enabling insertion, searching, and deletion.

The program supports inserting new records, searching by ID, deleting records, displaying records in sorted order, showing tree traversals, comparing balanced-looking and skewed insertion orders, and executing mandatory test cases.

Data Structure Representation

Each student is represented by a **Student** structure. A node has at most two children: a left child containing a smaller enrolment ID and a right child containing a larger ID.

Field	Purpose
enrolment_id	Unique integer key used to order and identify records
name	Student's full name
department	Student's department
left	Pointer to left subtree (smaller IDs)
right	Pointer to right subtree (larger IDs)

Ordering rule: All IDs in a node's left subtree are smaller; all IDs in its right subtree are greater. Duplicate enrolment IDs are rejected.

Initial Records

Ten sample records with IDs 101–110 are initialized in main(), providing a ready-made dataset for demonstrating BST operations.

Algorithms

Insertion

Insertion follows the BST ordering rule recursively. If the current pointer is null, a new node is created. If the ID already exists, insertion is rejected; otherwise the algorithm continues to the left or right subtree.

```
1. Start at the root.
2. If the node is NULL, create a new Student node.
3. If the ID already exists, reject the duplicate.
4. If ID < current ID, recurse left; otherwise recurse right.
```

Search

Search iteratively compares the target ID with the current node. It moves left for a smaller target and right for a larger target. Each visited-node comparison is counted and displayed.

```
1. Set current = root and comparisons = 0.
2. While current is not NULL, increment comparisons.
3. If key matches, display the record and stop.
4. If key is smaller, move left; otherwise move right.
5. If NULL is reached, report record not found.
```

Deletion

Deletion handles three cases: remove a leaf; replace a one-child node with its child; or, for a two-child node, copy the inorder successor (minimum of the right subtree) and delete that successor recursively.

```
1. Search for the requested ID.
2. Leaf: delete the node.
3. One child: return that child in its place.
4. Two children: copy inorder successor data.
5. Delete successor from the right subtree.
```

Tree Traversals

Inorder: Left → Root → Right (ascending IDs). **Preorder:** Root → Left → Right. **Postorder:** Left → Right → Root.

Sample Input / Output

Examples below use the records initialized by the program.

Display Records — Inorder

Enrolment ID	Name	Department
101	Ananya Gupta	CSE
102	Diya Patel	ECE
103	Kabir Singh	CE
104	Meera Joshi	IT
105	Aarav Sharma	CSE
106	Aditya Kumar	ME
107	Isha Nair	ECE
108	Rohan Verma	ME
109	Priya Reddy	IT
110	Siddharth Rao	CSE

Search Existing Record (ID 105)

```
--- Student Record Found ---
Enrolment ID : 105
Name         : Aarav Sharma
Department   : CSE
Total Comparisons Made: 1
```

Search Missing Record (ID 999)

```
Student with Enrolment ID 999 not found.
Total comparisons depend on the tree shape.
```

Duplicate Insertion (ID 105)

```
Error: Duplicate Enrolment ID (105) found! Record rejected.
```

Deletion Test Cases

Built-in tests demonstrate deletion of a leaf (101), a single-child node (110 in the test tree), and a two-child node (105), followed by an updated inorder display.

Efficiency Comparison

The program builds a balanced-looking tree and a sorted, skewed tree, then searches for ID 110 in each and prints comparison counts to show how tree shape affects performance.

Time and Space Complexity

Let **n** be the number of student records and **h** be the height of the BST. Operation costs depend on tree height, which varies with insertion order.

Operation	Average / balanced	Worst case (skewed)	Reason
Insertion	O(log n)	O(n)	One root-to-leaf path
Search	O(log n)	O(n)	May inspect nodes along height
Deletion	O(log n)	O(n)	Search plus successor handling
Inorder traversal	O(n)	O(n)	Visits each node once
Preorder traversal	O(n)	O(n)	Visits each node once
Postorder traversal	O(n)	O(n)	Visits each node once

Space complexity: O(n) to store n student nodes. Recursive insertion, deletion, and traversal use up to O(h) stack space; in a skewed tree, h may reach n.

Conclusion

The Student Record Search System demonstrates how a Binary Search Tree organizes student records by a unique enrolment ID. It implements insertion, search, deletion, and three standard traversals, along with duplicate-key handling and test cases. Operations are typically O(log n) in a balanced tree but can degrade to O(n) when insertion order creates a skewed tree. A self-balancing BST such as an AVL or Red-Black Tree could maintain logarithmic height for larger datasets.